

OFFLINEVAULT

Security Assessment Report

Plain-language security review of OfflineVault v1.0 (com.example.offlinevault)

Authorized local testing using dummy credentials and test data only

Assessment date: 3 October 2026

Executive summary

Overall result: the app has a solid basic encryption design, but important weaknesses were found around the master password and the locking workflow.

The most important issue is the visible **Lock** control. During testing, pressing Lock did not actually place the vault into a locked state. While the vault remained usable, the tester could still open a test entry and reveal its password without another master-password prompt.

Two other confirmed issues were the absence of a meaningful minimum master-password requirement (a one-character password was accepted) and the absence of an observed lockout/delay after 30 failed unlock attempts.

3	3	2	1
Confirmed findings	Important controls passed	Tests not fully conclusive	SQL-style bypass test failed

What worked well

The encrypted vault behaved as expected in our tests. SCrypt-based key derivation and AES-GCM decryption were verified locally. Backgrounding/reopening required the master password again. Screenshot protection and password masking also behaved as expected.

Who should read this?

This report is written so a non-technical owner can understand what happened, why it matters, and what should be fixed. Technical evidence is included for a developer who needs to reproduce the issues.

1. Quick navigation

- 2. Risk at a glance
- 3. Confirmed findings
- 4. Static and hardening observations
- 5. Security controls that worked
- 6. Test environment and methodology
- 7. Backup test and limitations
- 8. Recommended fixes
- 9. Final assessment

Assessment scope

The assessment focused on the local password-vault behavior visible in the tested APK and on practical checks that could be safely performed with dummy credentials. The goal was to validate the app rather than to attack any real account or real secret.

In scope	Out of scope
Vault unlock flow, password handling, lock behavior, backgrounding, screenshot exposure, clipboard/copy behavior, local encrypted storage, deletion, uninstall/reinstall, and a basic SQL-style bypass check.	Cloud backup because Backup Manager was disabled on the emulator; formal penetration testing against production infrastructure; recovery from physical memory; and attacks against real credentials.

2. Risk at a glance

Finding	Severity	Simple meaning
OFFV-01 - Lock button does not actually lock the vault	HIGH	A user may press Lock and reasonably believe secrets are protected, while the unlocked vault remains usable.
OFFV-02 - Extremely weak master passwords are accepted	HIGH	The app accepts a one-character master password, making weak user choices much easier to guess offline.
OFFV-03 - No lockout/delay observed after 30 failures	MEDIUM	Repeated guesses were not visibly restricted during the first 30 failed attempts tested.
Backup configuration needs validation	UNCONFIRMED	Backup settings raise a question, but the test device could not perform a real backup because Backup Manager was disabled.
Debuggable APK / release hygiene	CHECK	The tested APK was a debug build. Confirm the production build is not debuggable.

Important distinction

The report does **not** say that the AES-GCM encryption was broken. The demonstrated offline guessing result came from weak master-password choices, not from breaking the encryption algorithm.

3. Confirmed findings

HIGH

OFFV-01 - Lock button does not actually lock the vault

What we saw	Why it matters
Pressing the visible Lock control did not change the app into a locked state. The entry list stayed usable, and the test password could still be revealed.	A person with access to the open app may still be able to view stored secrets after pressing a control that appears to lock the vault. This creates a mismatch between what the user sees and the protection actually applied.

How we reproduced it

- 1) Unlock the vault. 2) Press Lock. 3) Open/View the dummy entry. 4) Press Show Password. 5) The password is displayed without another master-password prompt.

Evidence

Decompiled lifecycle code shows that on `ON_STOP` the app clears `vaultData`, `vaultPassword`, `masterPassword` and related UI state. A separate reset routine also clears `vaultData`, `vaultPassword` and `secret-display` state. This supports the intended lock behavior, but the exact button callback could not be proven from the large `Compose` method.

Testing limitation

The exact callback/wiring defect could not be proven statically because JADX failed to decompile the main `OfflineVaultScreen` method. The runtime failure itself was reproduced.

Recommended fix

Make the Lock control invoke the same effective state-clearing logic used when the app stops. Add a UI test that asserts: after Lock, View/Show Password cannot be used until unlock is completed again.

HIGH

OFFV-02 - Extremely weak master passwords are accepted

What we saw	Why it matters
A one-character master password (1) was accepted and successfully unlocked the vault.	The encryption is only as strong as the password protecting it. If someone obtains an encrypted vault, very weak passwords make offline guessing much easier.

How we reproduced it

Create a vault with a one-character password and unlock it successfully. Separately, create a dummy vault with 123456 and run the local verifier against a small dictionary; the weak password was recovered immediately. A stronger dummy password was not found by that same small dictionary.

Evidence

The local test PC performed 100 SCrypt derivations in about 13.035 seconds, around 7.67 guesses per second. This is hardware-specific and is included only as an observed benchmark, not as a universal attack speed.

Recommended fix

Enforce a strong minimum password/passphrase policy, reject obvious common passwords, and explain the rule clearly during vault creation.

MEDIUM

OFFV-03 - No lockout or delay observed after 30 failed unlock attempts

What we saw	Why it matters
Thirty incorrect unlock attempts were tested. The same error message appeared each time, with no observed temporary lockout, button disable, wipe, or increasing delay.	An attacker with the device available can keep trying password guesses. SCrypt slows each attempt, but the UI did not show an additional attempt-control mechanism during this test.

How we reproduced it

Attempt incorrect passwords repeatedly through the unlock screen. Tests were run up to 30 total failures, and the app continued returning the normal incorrect-password message.

Testing limitation

This proves behavior for the first 30 tested attempts only. It is not proof of unlimited attempts.

Recommended fix

Add progressive delays and/or temporary lockout after repeated failures, with a sensible recovery path for the legitimate owner.

4. Static and hardening observations

These items are important, but they are kept separate from the confirmed runtime findings so the report does not overstate what was proven.

Master password lifetime in memory

The decompiled code uses normal String values for password-related state. Ordinary immutable strings cannot be reliably wiped in place, so this is a hardening concern. The entered password is also passed into the vault operation as a normal string value. This was not demonstrated as a standalone exploit during the assessment.

Debug build / release hygiene

The tested APK was identified as application-debuggable. That is acceptable for a test build, but a production password-vault build should be verified to disable debugging and should have a release-build security gate.

Backup configuration - potential concern, not confirmed

The manifest inspection showed allowBackup=true and backup-rule resources. However, the emulator reported that Backup Manager was disabled, so a real backup/restore test was not possible. Do not report backup exposure as a confirmed vulnerability until it is tested on a backup-enabled environment.

What the code suggests about background protection

The decompiled lifecycle handler clears the vault contents, cached vault password, master password, confirmation password, entry fields, password-display state and other UI state when the lifecycle reaches ON_STOP. This matches the runtime result: after sending the app to the background and reopening it, the master password was required again.

5. Security controls that worked

Control	Result	Observation
Encryption at rest	PASS	The vault uses a binary format with a random salt and nonce. A local verifier successfully decrypted a known test vault with the correct master password and rejected incorrect passwords.
Background re-lock	PASS	After Home/background and reopen, the app requested the master password again. Decompiled lifecycle code also clears sensitive state on ON_STOP.
Screenshot/recent-app protection	PASS	Secure window flags were observed, and the recent-app preview did not expose the tested password screen.
Password masking	PASS	The password remained masked in Edit mode and normal text-selection/copy controls were not available.
Entry deletion	PASS	After deleting the dummy entry, the current vault decrypted to {"version":1,"entries":[]}. The deleted entry was not present in the current vault contents.
Uninstall cleanup	PASS	After uninstall/reinstall, the old local vault did not remain; the app started at the Create your encrypted vault screen.
SQL-style bypass	NO BYPASS	The payload ' OR '1'=1' -- did not bypass authentication and returned the normal incorrect-password message.

6. Test environment and methodology

Item	Value
App	OfflineVault v1.0
Package	com.example.offlinevault
Target SDK	37
APK	
Device	
Access	Authorized by app owner; dummy credentials and test data used

Test flow used

- 1 Create a dummy vault and dummy credential entry.
- 2 Check whether plaintext test secrets appear directly in the encrypted vault file.
- 3 Check logs for the dummy master password and dummy secret.
- 4 Test normal wrong-password behavior and a SQL-style bypass string.
- 5 Test very weak passwords, including a one-character password.
- 6 Test repeated failed unlock attempts up to 30 failures.
- 7 Test the visible Lock control and verify whether secret viewing remains possible.
- 8 Background the app and reopen it to test lifecycle re-locking.
- 9 Test password masking and normal copy behavior.
- 10 Uninstall and reinstall the app to observe local-data persistence.
- 11 Delete the dummy entry and decrypt the resulting vault to verify current contents.

Vault format observed

The vault file uses a binary structure containing a magic value (OVL_T), version, salt, nonce and ciphertext. The post-delete vault was successfully decrypted and contained an empty entries list.

7. Backup test and limitations

Backup configuration

Static inspection found `allowBackup=true` and both `backup_rules` and `data_extraction_rules` resources. The displayed XML-tree output did not show an exclusion entry for the vault file, which made backup behavior worth testing.

However, the emulator returned:

```
Backup Manager currently disabled
```

Conclusion for backup

Backup exposure is **not confirmed**. A real backup/restore test must be performed on a backup-enabled device or emulator before treating this as a security finding.

Other limitations

- Clipboard verification was limited by what the emulator exposed through system diagnostics, so absence of an observed clipboard value is not proof that no clipboard data could ever exist.
- The exact Lock-button callback was not proven from decompiled code because JADX could not completely decompile the large Compose screen method. The runtime behavior was nevertheless reproduced.
- The failed-attempt test covered the first 30 failures only.
- Deletion testing confirmed removal from the current active vault file. It did not attempt forensic recovery from filesystem remnants, snapshots, backups, or physical memory.

8. Recommended fixes

1. Fix the Lock control first

Make Lock immediately clear decrypted vault state, cached vault password, and secret-display state. After pressing Lock, every secret-view action should require unlock again.

2. Enforce a strong master-password policy

Reject very short/common passwords and provide a clear passphrase requirement. Consider a password-strength meter and a short explanation of why the rule exists.

3. Add failed-attempt protection

Use progressive delays or temporary lockout after repeated failures. Keep the recovery path usable for the legitimate owner.

4. Validate backup behavior

On a backup-enabled device, verify whether vault.enc is included in backup. If appropriate for the threat model, exclude the encrypted vault from device/cloud backup.

5. Harden the release build

Ensure production builds are not debuggable and add an automated release check for the final manifest and backup settings.

6. Reduce password lifetime in memory

Minimize how long the master password is held in UI state and use wipeable buffers for cryptographic operations where practical.

9. Final assessment

In simple words

The app **does encrypt the vault correctly in the tests we ran**. The bigger weaknesses are around the "front door": users can choose extremely weak master passwords, repeated guesses were not visibly restricted, and the visible Lock button did not actually lock the vault during testing.

Recommended priority for the developer

- 1 Fix the Lock button and verify it with an automated regression test.
- 2 Enforce a strong master-password policy and reject obviously weak choices.
- 3 Add repeated-attempt protection such as progressive delay/temporary lockout.
- 4 Validate backup behavior on a backup-enabled environment.
- 5 Re-run the same assessment after fixes and keep the test cases as part of the release checklist.

Assessment statement: This report reflects the behavior observed in the tested build on 3 October 2026 using authorized local testing and dummy data. Findings should be re-validated against the final production build because software behavior can change between builds.